AVALIAÇÃO · DESENVOLVIMENTO WEB · TESTES AUTOMATIZADOS

Exercício de testes unitários e de integração em JavaScript

Aluno: João Comini César de Andrade Turma: Ciência da Computação Data: 30 / 10 / 2026 Nota: _____ / 5,0

00

Instruções e critérios

Leia antes de responder — vale para todas as partes.

COMO RESPONDER

- **Parte I:** assinale **uma única** alternativa por questão. Questões com mais de uma marcação são anuladas.
- **Partes II e III:** escreva o código completo do teste — `describe`, `it` / `test`, asserções e hooks — e justifique as decisões pedidas no enunciado. Pseudocódigo só pontua se a estrutura de teste estiver correta.
- Considere Jest como runner padrão nas questões abertas. Quando o enunciado indicar Vitest, use `vi` no lugar de `jest`.
- Não é necessário instalar nada: avalie a lógica dos testes, não a saída do terminal.

DISTRIBUIÇÃO DA PONTUAÇÃO

PARTE	ITENS	VALOR UNITÁRIO	SUBTOTAL
I — Múltipla escolha	10	0,1	1,0
II — Testes unitários (abertas)	2	1,0	2,0
III — Testes de integração (abertas)	2	1,0	2,0
Total	14	—	**5,0**

O QUE MAIS CONTA NA CORREÇÃO DAS ABERTAS

- Um teste, uma responsabilidade — nomes descritivos em `it` / `test`.
- Estrutura AAA visível (Arrange, Act, Assert), mesmo sem comentários.
- Asserções sobre **comportamento**, não sobre implementação interna.
- Isolamento: estado limpo entre testes, mocks restaurados/limpos nos hooks.
- Cobrir o caminho de erro, não só o caminho feliz.

LEMBRETE CONCEITUAL

Teste unitário exercita uma unidade isolada (função, classe, módulo) sem dependências externas reais — rápido, determinístico e preciso ao apontar a falha. **Teste de integração** exercita várias partes funcionando juntas (módulo + DOM, rota + banco, serviço + API) — mais lento, porém valida o fluxo real. A pirâmide recomenda muitos unitários, menos integração e poucos E2E.

01

Parte I — Múltipla escolha

Dez questões · 0,1 ponto cada · 5 pontos no total · uma alternativa correta.

QUESTÃO 01 · CONCEITOS

Sobre a diferença entre testes unitários e de integração, qual afirmação está **correta**?

A. Testes de integração são mais rápidos que os unitários porque não precisam montar o ambiente de DOM.

B. Testes unitários devem usar as dependências reais de rede para produzir maior confiança.

C. Testes unitários isolam a unidade com mocks e costumam apontar a linha exata da falha; testes de integração validam o fluxo completo e são mais lentos.

D. A pirâmide de testes recomenda mais testes E2E do que unitários, pois E2E cobre mais código.

QUESTÃO 02 · PADRÃO AAA

Considere o teste abaixo:

```
test('formata valor em BRL corretamente', () => {
  const value = 1500                    // (1)
  const result = formatCurrency(value)  // (2)
  expect(result).toBe('R$ 1.500,00')    // (3)
})
```

A linha **(2)** corresponde a qual etapa do padrão AAA?

A. Arrange — preparação dos dados de entrada.

B. Act — execução da unidade sob teste.

C. Assert — verificação do resultado esperado.

D. AfterEach — limpeza do estado após o teste.

QUESTÃO 03 · MATCHERS

Um teste precisa comparar o array de itens do carrinho com `[]` e outro precisa comparar um objeto de usuário retornado pela API com o objeto esperado. Qual par de decisões está correto?

A) `toBe` faz comparação profunda de objetos e arrays; `toEqual` usa igualdade estrita (`===`).

B) `toBe` usa igualdade estrita (`===`), adequada a primitivos; `toEqual` compara a estrutura de objetos e arrays, sendo o matcher correto para ambos os casos.

C) Os dois matchers são intercambiáveis para arrays, mas `toEqual` falha com objetos aninhados.

D) `toContain` é o matcher indicado tanto para comparar arrays quanto objetos completos.

QUESTÃO 04 · ERROS

O método `cart.add({ name: 'Produto' })` deve lançar o erro `'Preço obrigatório'`. Qual asserção verifica isso **corretamente**?

A) `expect(cart.add({ name: 'Produto' })).toThrow('Preço obrigatório')`

B) `expect(() => cart.add({ name: 'Produto' })).toThrow('Preço obrigatório')`

C) `await expect(cart.add({ name: 'Produto' })).rejects.toThrow('Preço obrigatório')`

D) `expect(cart.add).toThrow('Preço obrigatório')`

QUESTÃO 05 · HOOKS E ISOLAMENTO

Na suíte da classe `Cart`, cada teste começa com:

```
let cart
beforeEach(() => {
  cart = new Cart()
})
```

Qual é a finalidade desse `beforeEach`?

A) Tornar a execução mais rápida, reaproveitando a mesma instância entre os testes.

B) Garantir isolamento: cada teste recebe um carrinho vazio, sem depender da ordem de execução nem de efeitos colaterais de testes anteriores.

C) É obrigatório pelo Jest em toda suíte que declara um `describe`.

D) Limpar automaticamente os mocks criados com `jest.fn()`.

QUESTÃO 06 · TESTING LIBRARY

Segundo a hierarquia de prioridade das queries da Testing Library, qual estratégia de seleção está **correta**?

A `getByTestId` tem prioridade máxima e deve ser a primeira escolha em qualquer componente.

B `getByPlaceholderText` é a query de prioridade máxima, por ser a mais específica.

C Priorizar `getByRole` e `getByLabelText`, que encontram elementos como usuários reais fazem, e usar `getByTestId` apenas como último recurso.

D Usar `document.querySelector` com classes CSS, pois reflete melhor a estrutura real do DOM.

QUESTÃO 07 · ASSINCRONISMO

No teste de login, após o clique em “Entrar”, a asserção é envolvida por `waitFor`:

```
await waitFor(() => {
  expect(screen.getByText(/bem-vindo/i)).toBeInTheDocument()
})
```

Por que esse envolvimento é necessário?

A Porque `getByText` é uma função assíncrona e precisa de `await` direto.

B Porque a atualização do DOM depois do clique e da resposta da API é assíncrona; o `waitFor` repete a asserção até ela passar ou até estourar o timeout.

C Porque o Jest exige `waitFor` em todo teste que use `fireEvent`.

D Para forçar uma pausa fixa de um segundo e evitar testes intermitentes.

QUESTÃO 08 · MOCK SERVICE WORKER

Qual afirmação descreve corretamente o papel do MSW nos testes de integração?

A Ele substitui o módulo importado pelo componente, o que exige alterar o código de produção para injetar a dependência.

B Ele intercepta `fetch` e `axios` na camada de rede, de modo que o código de produção continua chamando `fetch('/api/...')` normalmente, sem saber que está sendo mockado.

C Ele só funciona em navegadores reais, sendo incompatível com Jest e jsdom.

D Ele gera o relatório de cobertura de código no lugar do Jest.

QUESTÃO 09 · API REST

No teste de integração de `POST /usuarios`, um usuário já existe com o e-mail `ana@email.com`. Ao enviar um segundo cadastro com o mesmo e-mail, o teste deve esperar:

A) Status `200`, sobrescrevendo o usuário existente.

B) Status `201`, criando um registro duplicado.

C) Status `409`, com o corpo do erro indicando duplicidade (por exemplo, casando com `/duplicado/i`).

D) Status `500`, pois o banco lança uma exceção não tratada.

QUESTÃO 10 · AUTENTICAÇÃO

No fluxo de autenticação JWT, o teste acessa `GET /perfil` **sem** o header `Authorization`. O status esperado e a razão correta são:

A) `200`, pois `/perfil` devolve apenas dados públicos.

B) `302`, redirecionando para a tela de login.

C) `409`, porque falta um dado obrigatório na requisição.

D) `401`, pois a rota protegida exige um token válido e a ausência do header caracteriza requisição não autenticada.

02

Parte II — Testes unitários

Duas questões abertas · 1,0 ponto cada · escreva o código do teste.

QUESTÃO 11 · UNITÁRIO · 1,0 PONTO

Função pura — `calcularTotal`

O módulo `carrinho.js` exporta a função abaixo:

```
// src/utils/carrinho.js
export function calcularTotal(itens) {
  return itens.reduce((total, item) => {
    if (typeof item.preco !== 'number') {
      throw new Error('Preço inválido')
    }
    return total + item.preco
  }, 0)
}
```

Escreva uma suíte de testes unitários (`carrinho.test.js`) que cubra, no mínimo: **(a)** a soma correta dos preços de um carrinho com três itens; **(b)** o carrinho vazio; **(c)** o lançamento de `'Preço inválido'` quando um item não tem preço numérico.

Use o padrão AAA, nomes descritivos e os matchers adequados. Em seguida, responda em duas ou três linhas: *por que este teste não precisa de DOM, de mocks ou de banco de dados, e por que ele é mais rápido e mais preciso que um teste de integração equivalente?*

```
// carrinho.test.js
import { calcularTotal } from '../src/utils/carrinho.js'

describe('calcularTotal', () => {
  it('soma os preços de um carrinho com três itens', () => {
    // Arrange
    const itens = [{ preco: 10 }, { preco: 20.5 }, { preco: 4.5 }]
    // Act
    const total = calcularTotal(itens)
    // Assert
    expect(total).toBe(35)
  })

  it('retorna 0 para um carrinho vazio', () => {
    const itens = [] // Arrange
    const total = calcularTotal(itens) // Act
    expect(total).toBe(0) // Assert
  })

  it("lança 'Preço inválido' quando o item não tem preço numérico", () => {
    const itens = [{ preco: 10 }, { nome: 'Caneta' }] // Arrange
    const acao = () => calcularTotal(itens) // Act
    expect(acao).toThrow('Preço inválido') // Assert
  })
})
```

Justificativa: calcularTotal é uma função pura: depende só do argumento e devolve um valor, sem ler DOM, rede ou banco; basta passar um array e comparar o retorno, então não há nada para mockar. Por isso o teste roda em milissegundos, é determinístico e, se falhar, aponta direto para a lógica do reduce; um teste de integração equivalente montaria ambiente e várias camadas, ficando mais lento e com a causa da falha mais difícil de localizar.

QUESTÃO 12 · UNITÁRIO · 1,0 PONTO

Dependência mockada — `buscarUsuario`

O serviço abaixo depende do módulo `api.js`, que faz a requisição HTTP:

```
// src/services/usuarioService.js
import * as api from './api.js'

export async function buscarUsuario(id) {
  const { data } = await api.get(`/usuarios/${id}`)
  return data
}
```

Escreva os testes unitários de `buscarUsuario` cobrindo **sucesso** e **falha da API**. O teste de sucesso deve substituir `api.get` por um mock que resolve `{ data: { id: 1, nome: 'Ana' } }`, verificar que foi chamado com `'/usuarios/1'` e que o retorno é o objeto do usuário. O teste de falha deve fazer o mock rejeitar com `new Error('404')` e verificar a rejeição da Promise.

Explique também, em uma frase, por que é necessário limpar ou restaurar o spy entre os testes.

```
// usuarioService.test.js
import * as api from '../src/services/api.js'
import { buscarUsuario } from '../src/services/usuarioService.js'

describe('buscarUsuario', () => {
  let getSpy

  beforeEach(() => {
    getSpy = jest.spyOn(api, 'get')
  })

  afterEach(() => {
    jest.restoreAllMocks()
  })

  it('retorna o usuário quando a API responde com sucesso', async () => {
    // Arrange
    getSpy.mockResolvedValue({ data: { id: 1, nome: 'Ana' } })
    // Act
    const usuario = await buscarUsuario(1)
    // Assert
    expect(getSpy).toHaveBeenCalledWith('/usuarios/1')
    expect(usuario).toEqual({ id: 1, nome: 'Ana' })
  })

  it('rejeita a Promise quando a API falha', async () => {
    // Arrange
    getSpy.mockRejectedValue(new Error('404'))
    // Act + Assert
    await expect(buscarUsuario(1)).rejects.toThrow('404')
    expect(getSpy).toHaveBeenCalledWith('/usuarios/1')
  })
})
```

Justificativa: sem restaurar/limpar o spy, o mock e o histórico de chamadas do teste anterior vazam para o seguinte (api.get continuaria substituída e toHaveBeenCalledWith enxergaria chamadas antigas), quebrando o isolamento e fazendo o resultado depender da ordem de execução.

03

Parte III — Testes de integração

Duas questões abertas · 1,0 ponto cada · interface e API.

QUESTÃO 13 · INTEGRAÇÃO · 1,0 PONTO

Fluxo de login na interface

O componente `loginForm.js` renderiza um formulário com os campos rotulados **E-mail** e **Senha**, um botão com o nome acessível **Entrar** e, após o envio, chama a API. Em caso de sucesso, exibe o texto “Bem-vindo, …”. Em caso de credenciais inválidas, exibe um elemento com `role="alert"` contendo “Credenciais inválidas”.

Escreva o teste de integração (`login.test.js`) com Testing Library, cobrindo **os dois cenários**. O teste deve renderizar o formulário e preencher os campos por `getByLabelText` , clicar no botão localizado por `getByRole('button', { name: 'Entrar' })` , usar `waitFor` nas asserções que dependem da resposta assíncrona e respeitar o isolamento entre os testes (limpeza do DOM e reset dos handlers).

Justifique, em duas ou três linhas, por que a query por `role` / `label` é preferível a `getByTestId` neste caso, e por que este teste é de integração e não unitário.

```
// login.test.js  (React Testing Library + MSW)
import { render, screen, waitFor, cleanup } from '@testing-library/react'
import userEvent from '@testing-library/user-event'
import '@testing-library/jest-dom'
import { http, HttpResponse } from 'msw'
import { setupServer } from 'msw/node'
import LoginForm from '../src/loginForm.js'
const server = setupServer()
beforeAll(() => server.listen())
afterEach(() => { server.resetHandlers(); cleanup() }) // reset dos handlers + limpeza do DOM
afterAll(() => server.close())
async function entrar(email, senha) { // etapa Act compartilhada
  await userEvent.type(screen.getByLabelText('E-mail'), email)
  await userEvent.type(screen.getByLabelText('Senha'), senha)
  await userEvent.click(screen.getByRole('button', { name: 'Entrar' }))
}
describe('Fluxo de login', () => {
  it('exibe "Bem-vindo, Ana" com credenciais válidas', async () => {
    // Arrange
    server.use(http.post('*/api/login', () => HttpResponse.json({ nome: 'Ana' })))
    render(<LoginForm />)
    // Act
    await entrar('ana@email.com', 'senha123')
    // Assert
    await waitFor(() => {
      expect(screen.getByText(/bem-vindo, ana/i)).toBeInTheDocument()
    })
  })
  it('exibe alerta "Credenciais inválidas" quando a API recusa o login', async () => {
    // Arrange
    server.use(http.post('*/api/login', () =>
      HttpResponse.json({ erro: 'Credenciais inválidas' }, { status: 401 })))
    render(<LoginForm />)
    // Act
    await entrar('ana@email.com', 'errada')
    // Assert
    await waitFor(() => {
      expect(screen.getByRole('alert')).toHaveTextContent(/credenciais inválidas/i)
    })
  })
})
```

Justificativa: getByRole e getByLabelText localizam os elementos como o usuário os percebe (rótulo e nome acessível), validam de quebra a acessibilidade e sobrevivem a mudanças de marcação; getByTestId é último recurso, pois acopla o teste a um atributo que o usuário não vê. É integração porque exercita juntos formulário, estado do componente, requisição HTTP (interceptada pelo MSW na camada de rede) e atualização do DOM com a resposta, e não uma unidade isolada.

QUESTÃO 14 · INTEGRAÇÃO · 1,0 PONTO

API REST com banco de teste

Uma API Express expõe `POST /usuarios` (campos `nome` e `email`). O e-mail é único: um cadastro duplicado responde `409` com corpo contendo a palavra "duplicado". Os testes usam Supertest e um banco SQLite de teste, com o ciclo de vida:

```
beforeAll(async () => { await db.migrate.latest() })
afterEach(async () => { await db('usuarios').truncate() })
afterAll(async () => { await db.destroy() })
```

Escreva a suíte de integração (`usuarios.integration.test.js`) cobrindo: **(a)** criação com dados válidos → `201`, corpo com o nome enviado e `id` definido; **(b)** e-mail duplicado → `409` e corpo casando com `/duplicado/i`.

Explique, em um parágrafo curto: *por que truncar a tabela no `afterEach` é indispensável para o teste (b) passar de forma confiável, e por que não se deve mockar o banco nesta suíte.*

```
// usuarios.integration.test.js
import request from 'supertest'
import app from '../src/app.js'
import db from '../src/db.js'

beforeAll(async () => { await db.migrate.latest() })
afterEach(async () => { await db('usuarios').truncate() })
afterAll(async () => { await db.destroy() })

describe('POST /usuarios', () => {
  it('cria o usuário com dados válidos e responde 201', async () => {
    // Arrange
    const payload = { nome: 'Ana', email: 'ana@email.com' }
    // Act
    const res = await request(app).post('/usuarios').send(payload)
    // Assert
    expect(res.status).toBe(201)
    expect(res.body.nome).toBe('Ana')
    expect(res.body.id).toBeDefined()
  })

  it('responde 409 ao cadastrar e-mail duplicado', async () => {
    // Arrange: usuário já existente
    const payload = { nome: 'Ana', email: 'ana@email.com' }
    await request(app).post('/usuarios').send(payload)
    // Act: segundo cadastro com o mesmo e-mail
    const res = await request(app).post('/usuarios').send(payload)
    // Assert
    expect(res.status).toBe(409)
    expect(JSON.stringify(res.body)).toMatch(/duplicado/i)
  })
})
```

Justificativa: sem o truncate no afterEach, o registro criado em um teste permanece no banco e contamina o seguinte: o teste (a), se rodar depois de outro que gravou ana@email.com, receberia 409 em vez de 201, e o (b) poderia passar mesmo que seu próprio Arrange não tivesse funcionado, escondendo defeitos; o resultado passaria a depender da ordem de execução. Com a tabela vazia, cada teste monta o próprio cenário (o (b) cria explicitamente o primeiro usuário) e o resultado é determinístico. O banco não deve ser mockado porque o 409 nasce da restrição de unicidade real e da migration; um mock apenas reimplementaria a regra que se quer verificar e deixaria passar erros de schema, constraint ou query, anulando o propósito do teste de integração.